

Università di Bologna
Corso di Laurea in Tecnologie dei Sistemi Informatici
Progettazione e Sviluppo Software — A.A. 2025/2026

Relazione di Progetto

Memory Game

Componenti del gruppo:

Marco Toma — Manuel Tomasello

marco.toma3@studio.unibo.it — manuel.tomasello@studio.unibo.it

Repository: <https://github.com/manueltomasello/PSS25-memorygame>

Indice

Capitolo 1 – Analisi

1.1 Descrizione e Requisiti

1.2 Modello del Dominio (UML)

Capitolo 2 – Design

2.1 Architettura (Model-View-Controller)

2.2 Design Dettagliato

2.2.1 Marco Toma

2.2.2 Manuel Tomasello

2.2.3 Manuel Tomasello (ScoreManager)

2.2.4 Manuel Tomasello (MenuPanel)

2.2.5 Manuel Tomasello (StatoPanel)

2.2.6 Manuel Tomasello (EndGameDialog)

Capitolo 3 – Sviluppo

3.1 Testing Automatizzato (JUnit)

3.2 Note di sviluppo (Permalink GitHub)

3.2.1 Marco Toma (Timer)

3.2.2 Manuel Tomasello

Capitolo 4 – Commenti finali

4.1 Autovalutazione

4.1.1 Marco Toma

4.1.2 Manuel Tomasello

4.2 Commenti finali

Appendice A - Guida Utente

- *Avvio del gioco e selezione difficoltà*
- *Interfaccia di gioco e svolgimento*
- *Fine partita e messaggi di vittoria*

Appendice B – Consuntivo Ore

- *B.0.1 Marco Toma*
- *B.0.2 Manuel Tomasello*

Capitolo 1 Analisi

1.1 Descrizione e Requisiti

Abbiamo pensato di realizzare un software in particolare un videogioco un classico, il Memory con interfaccia grafica. Il giocatore deve trovare tutte le coppie di carte uguali presenti su una griglia, girandone due alla volta. Se le carte corrispondono rimangono scoperte; in caso contrario vengono rigirate dopo un breve intervallo. La partita termina quando tutte le coppie sono state trovate.

Requisiti funzionali

- Il sistema permette di selezionare il livello di difficoltà prima di iniziare la partita (facile, media, difficile), che determina le dimensioni della griglia di gioco.
- Il sistema mostra una griglia di carte coperte mescolate casualmente all'inizio di ogni partita.
- Il giocatore può girare due carte per turno; se corrispondono rimangono scoperte, altrimenti vengono richiuse automaticamente dopo un breve ritardo.
- Il sistema tiene traccia in tempo reale del numero di mosse effettuate e delle coppie trovate.
- La partita termina con un messaggio di vittoria quando tutte le coppie sono state individuate.
- Al termine della partita viene mostrato un riepilogo con il numero di mosse e una citazione casuale.

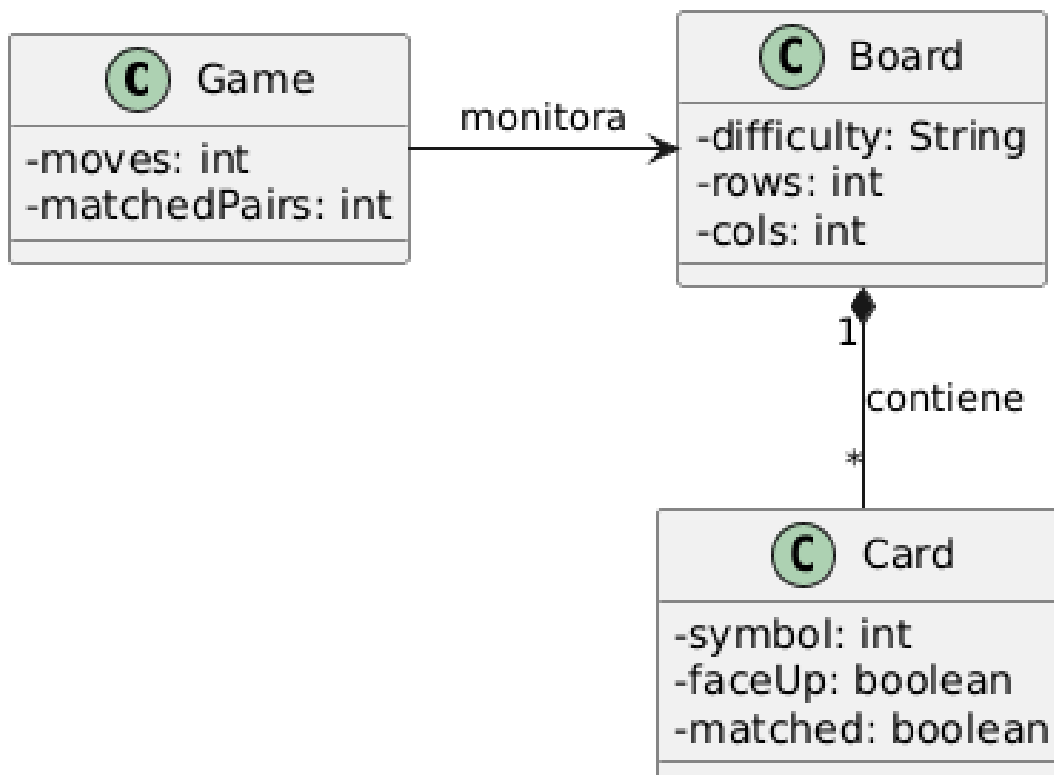
Requisiti non funzionali

- Portabilità: il software deve essere eseguibile su Windows e Linux tramite un unico file JAR autonomo, senza risorse esterne.
- Build automatizzata: il progetto deve essere compilabile e testabile con un singolo comando Gradle.
- Correttezza dell'encoding: tutti i testi devono essere visualizzati correttamente su tutte le piattaforme (UTF-8).

1.2 Modello del Dominio

Il dominio del Memory ruota attorno al concetto di carta (Card): ogni carta è caratterizzata da un identificativo numerico (simbolo) che definisce l'appartenenza alla coppia. Lo stato della carta è gestito tramite due flag booleani: uno per la visibilità (faceUp) e uno per l'avvenuto abbinamento (matched).

Una partita (Game) tiene traccia dell'avanzamento: conta le mosse effettuate dal giocatore e le coppie trovate, determinando la condizione di vittoria quando tutte le coppie sono state abbinare. Il giocatore interagisce con la Board girando le carte; il sistema gestisce autonomamente la logica di confronto e chiusura.



Capitolo 2 Design

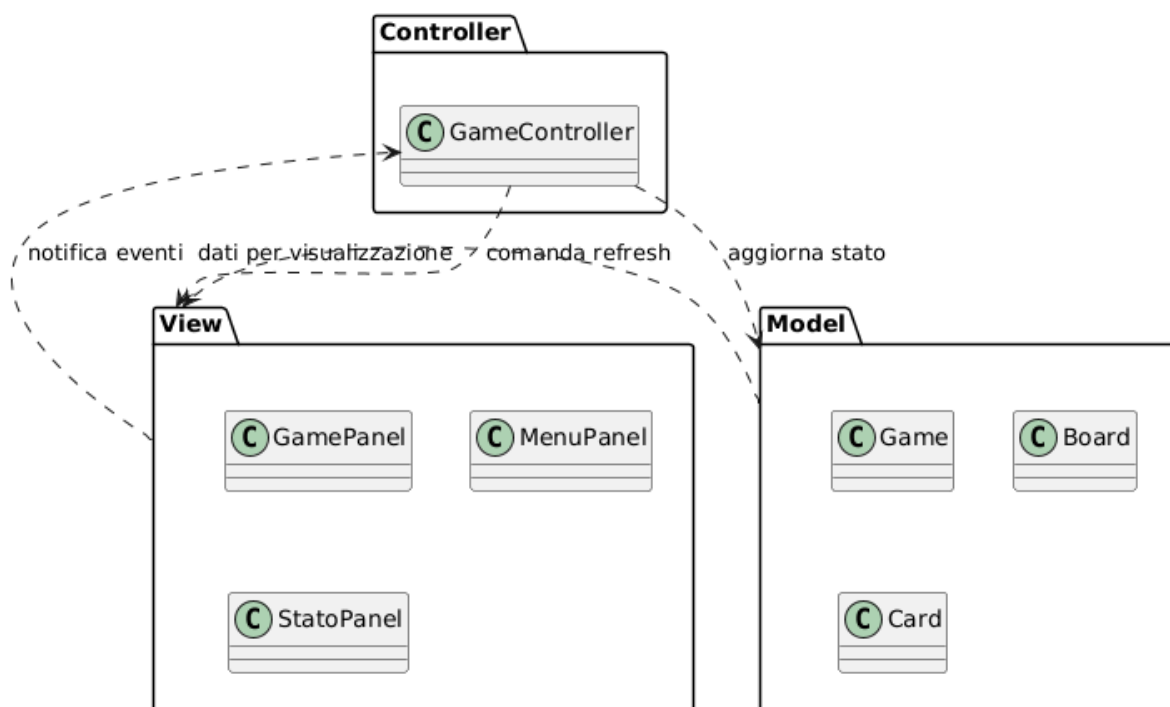
2.1 Architettura

L'architettura del software si basa sul pattern Model-View-Controller (MVC), una scelta fondamentale per garantire la manutenibilità e la separazione delle responsabilità.

- Model (Card, Board, Game): Rappresenta il "magazzino" del sistema, queste classi gestiscono lo stato interno del gioco (valore delle carte, conteggio delle mosse, stato della griglia) senza avere alcuna conoscenza della parte grafica.
- View (Pacchetto gui): È l'interfaccia visiva con cui l'utente interagisce cattura gli input (come il click su una tessera) ma non possiede la logica per decidere se una mossa è valida o meno.
- Controller (GameController): Funge da mediatore e il "cervello" dell'applicazione.

Il flusso di esecuzione segue una logica precisa: Quando l'utente clicca su una carta nella View, l'evento viene immediatamente inoltrato al Controller. Il Controller interroga il Model per verificare lo stato della carta e la validità della mossa. Una volta ricevuta conferma dai dati, il Controller impartisce alla View l'ordine di aggiornarsi graficamente (ad esempio, mostrando il simbolo della carta o colorando la coppia trovata).

Questo approccio garantisce il disaccoppiamento se in futuro volessimo cambiare la libreria grafica o il tipo di simboli usati, la logica di controllo e la struttura dei dati rimarrebbero invariate.



"L'architettura del sistema si basa sul modello di dominio descritto nel Capitolo 1 e segue il pattern Model-View-Controller per separare la logica dalla visualizzazione."

2.2 Design Dettagliato

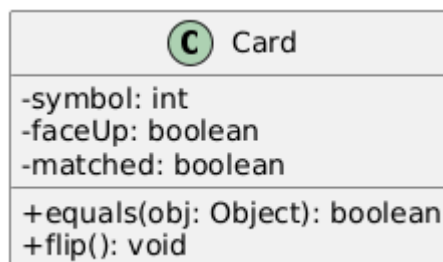
2.2.1 Marco Toma

Per gestire la chiusura delle carte quando la coppia è sbagliata, ho scelto di non usare un comando che blocca il programma (come lo sleep), perché questo avrebbe congelato tutta la grafica.

Ho scelto invece di usare un `javax.swing.Timer`, che permette di aspettare mezzo secondo lasciando il gioco fluido; per evitare che l'utente clicchi altre carte in quel mezzo secondo ho usato una variabile di controllo (un flag booleano).

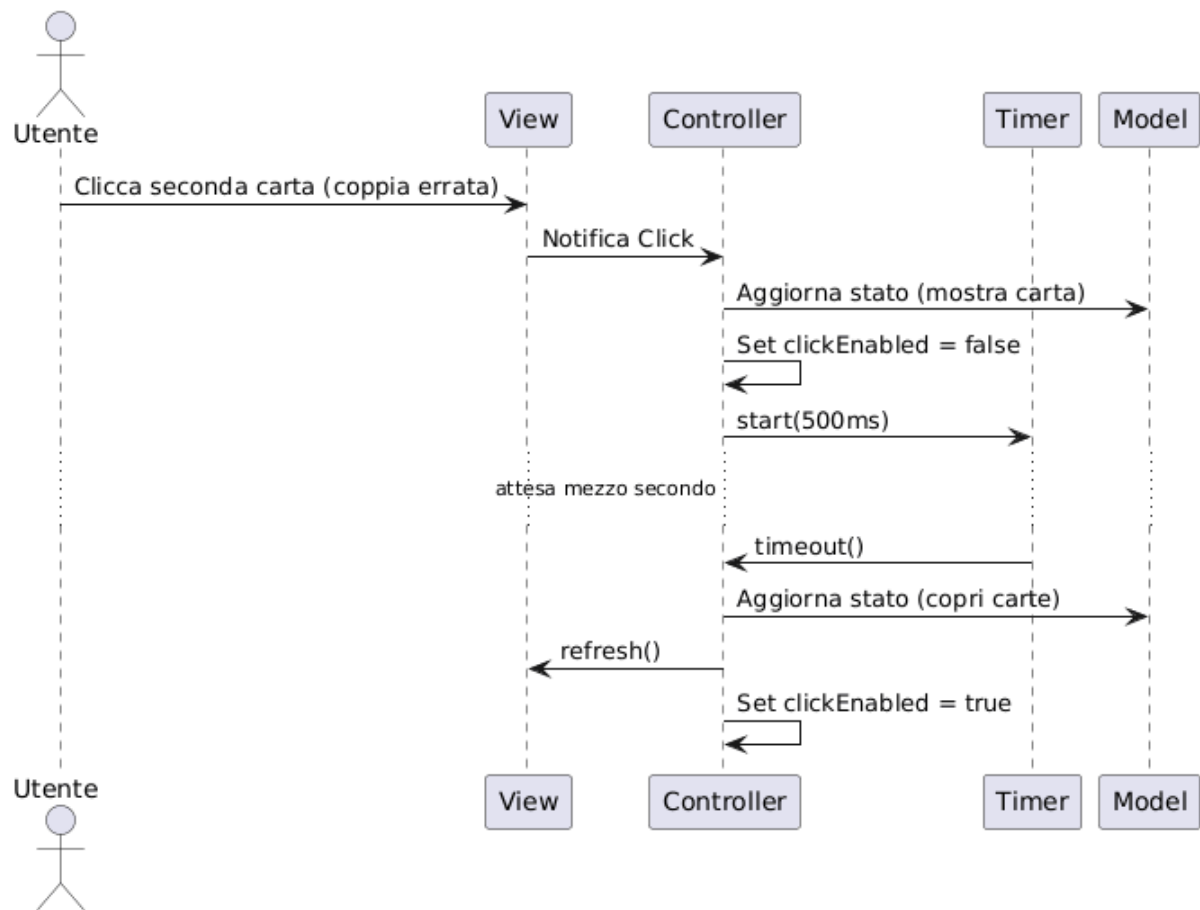
Il sistema non confronta i pixel dell'immagine, perché sarebbe troppo pesante e lento, ma uso il metodo `equals()` per fare un controllo, lo riassumo in tre punti:

- 1) L'Identità: Ogni carta è un oggetto a sé perché ha una posizione diversa sulla griglia;
- 2) Il Contenuto: Ogni carta ha un attributo chiamato `symbol` (un numero intero), anche se le carte sono in posizioni diverse, due carte "gemelle" avranno lo stesso identico numero salvato in questa variabile ovvero il simbolo della carta;
- 3) Il Metodo `equals()`: Uso questo metodo per istruire il Controller a ignorare il fatto che le carte sono oggetti distinti, gli dico: "Guarda solo il `symbol`: se il numero è lo stesso, allora la coppia è corretta.



Problema: gestione del turno e blocco input — Quando due carte non corrispondono, l'utente non deve poter cliccare altre carte mentre le prime due sono ancora visibili (in attesa della chiusura automatica).

Soluzione: introduzione di un flag booleano `waiting` nel `GameController`. Quando attivo, il metodo `onCardClicked()` esce immediatamente. Un `javax.swing.Timer` viene usato per scatenare la chiusura delle carte dopo 500ms e resettare il flag. Questo evita race conditions logiche senza l'uso di thread aggiuntivi.



2.2.2 Manuel Tomasello

La classe Board è il tabellone di gioco del nostro memory-game, dove vengono disposte le coppie di carte, la quantità è variabile in funzione della difficoltà.

Gli attributi contenuti nella board sono:

- 1) Cards: un array che contiene le carte;
- 2) Rand: un generatore di numeri randomici usato per mescolare le carte;
- 3) Difficulty: che riceve un oggetto Difficulty un enum pubblico dichiarato in Difficulty.java, esso può assumere come valore 3 matrici, easy(4x4), Medium (6x6), Hard (8x8).

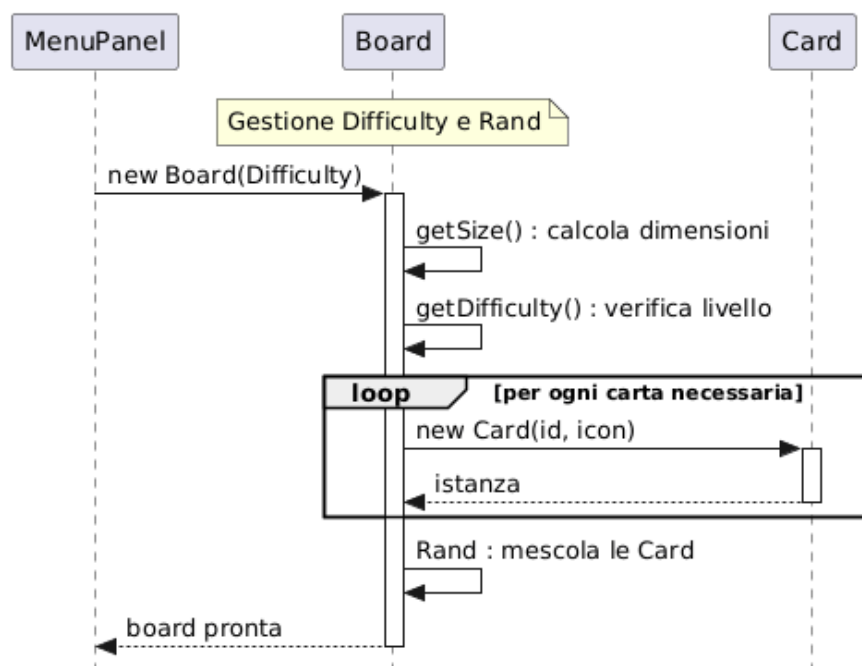
Il costruttore riceve Difficulty, calcola il numero totale di carte e chiama genCards(), ovvero il cuore della board che lavora in due fasi.

Nella prima crea le coppie di carte assegnando a ciascuna un numero intero progressivo, ovviamente viene ripetuto due volte nell'array (le coppie), nella seconda fase avviene il mescolamento con l'algoritmo di Fisher-Yates.

Nel secondo ciclo for si scorre per l'array fino alla fine, per ogni posizione "i" viene scelta una posizione casuale "j", ovviamente mi servo di una variabile di appoggio per scambiare gli indici che in questo caso ho chiamato temp.

Gli altri metodi sono:

- 1) getCard() che restituisce la carta alla posizione richiesta;
- 2) getSize() che restituisce il numero totale di carte;
- 3) get Difficulty() che restituisce la difficoltà;



2.2.3 ScoreManager

ScoreManager è una classe che gestisce i punteggi, ovvero il numero di mosse effettuate per scoprire tutte le card della board. Il punteggio migliore viene salvato e ad ogni partita della stessa difficoltà e viene comparato.

Il punteggio viene salvato dentro un file ([.memory-pss-scores.properties](#)) nella home dell'utente, questo meccanismo funziona sia su Linux che su Windows all'interno vi possono essere un massimo di 3 stringhe con i punteggi e la difficoltà relativa.

Metodi:

GetBestScore() → legge il punteggio della difficoltà selezionata, se non esiste ancora nessun record allora restituisce il primo punteggio effettuato che per forza di cose è il massimo intero possibile.

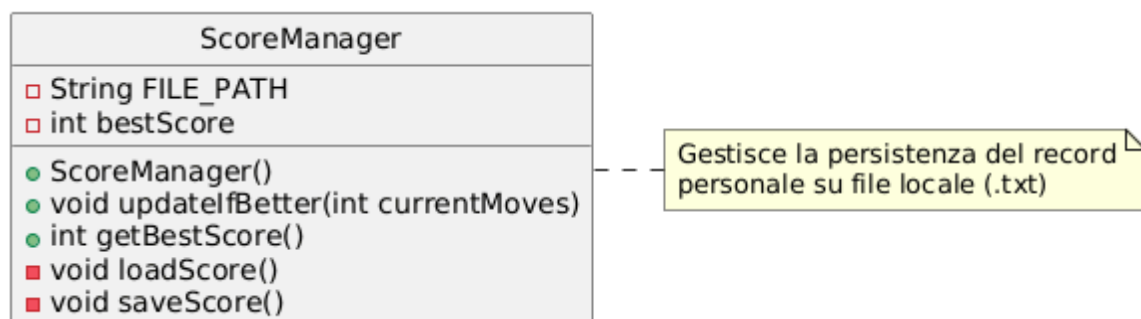
UpdateIfBetter() → all'interno di questo metodo viene effettuato un confronto tra le mosse effettuate e il punteggio della difficoltà in essere, se restituisce true vuol dire che il file è stato aggiornato con il nuovo record, altrimenti restituisce false.

GetBestScoreText() → mi restituisce una stringa che passo alla UI più nello specifico la EndGameDialog, all'interno di questo metodo viene effettuato un confronto tra il miglior punteggio associato della difficoltà passata come parametro e integer.Max_Value che se uguali mi permette di sapere se non ci sono registrati record.

Riassumendo al primo avvio dato che non vi è uno storico, verrà restituito: "Nessun Record" le volte successive restituirà il numero delle mosse.

load() → è un metodo chiamato dal costruttore che carica il file, se non esiste per evitare di mandare il programma in crash viene in aiuto la struttura try catch, molto utile per gestire gli eventi come questo.

Save() → chiamato nel metodo updateIfBetter mi permette di andare a sovrascrivere il file che contiene i record.



2.2.4 MenuPanel

MenuPanel (BorderLayout)

MenuPanel è il pannello iniziale del gioco memory, realizzato grazie alle librerie grafiche java swing.

Il layout JPanel è composto da un BorderLayout che è il contenitore principale, viene suddiviso in 3 parti:

North → contiene il titolo del gioco;

Center → contiene quattro sotto pannelli;

South → contiene il pulsante di avvio partita;

Nel Center la prima cosa che si incontra sono tre JRadioButton che permettono di aggiornare il campo selectedDifficulty con i valori dell'enum Difficulty precedentemente descritti.

Come seconda opzione si dà la possibilità all'utente di scegliere il tema tra chiaro e scuro, vi sono poi le regole del gioco in una JTextArea non editabile e infine i crediti in una JLabel.

Il Pannello riceve il MainFrame come dipendenza del costruttore, ciò mi permette di invocare i metodi pubblici (ApplicaTema e AvviaPartita) sul frame senza che il menù conosca i dettagli dell'applicazione.



2.2.5 StatoPanel

StatoPanel

StatoPanel è un pannello di stato che mostra in tempo reale le informazioni sulla partita in corso, una sorta di head up display, è strutturato per estendere JPanel e sfrutta un FlowLayout centrato in modo tale che le etichette siano disposte in fila in modo ordinato, i tre JPanel che contiene sono:

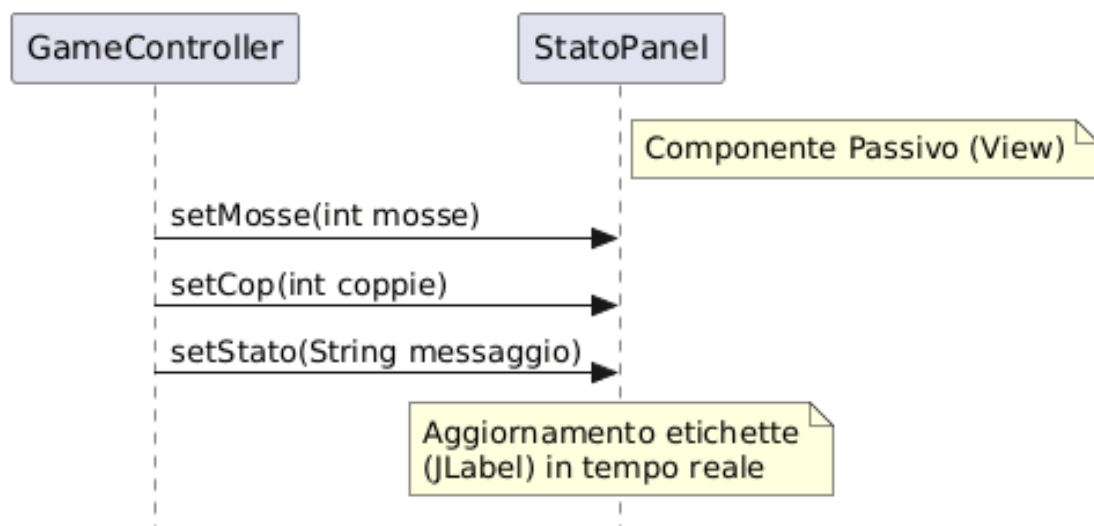
MosseLabel → conta il numero di coppie scoperte in tempo reale;

CoppieLabel → mostra le coppie trovate sul totale sempre in tempo reale;

StatoLabel → è inizializzato a spazio vuoto ma contiene dei messaggi di stato (vittoria errore);

I campi sono private final, quindi non modificabili dall'esterno, di fatto l'aggiornamento avviene attraverso tre metodi pubblici: setMosse, setCop, setStato.

Per quanto riguarda l'architettura statoPanel non contiene logiche di gioco è un componente strettamente passivo, si aggiorna solo quando è il controller a volerlo, questo evidenzia una separazione netta tra view e logica applicativa.



2.2.6 EndGameDialog

EndGameDialog

EndGameDialog è la finestra di dialogo mostrata al termine della partita, implementata estendendo JDialog di Swing.

Riceve sei parametri:

Jframe parent → è la finestra padre che viene utilizzata per centrare il dialogo e per la modalità modale, questo vuol dire che io devo interagire con questa finestra altrimenti non posso andare avanti.

Difficulty difficulty e int moves → sono i dati della partita appena giocata;

ScoreManager scoreManager → gestore dei record (precedentemente descritto);

Runnable onNewGame e onMenu, callback che gestisce il comportamento dei pulsanti con Runnable ciò permette di avere un dialogo che non sa cosa accade dopo la chiusura, runnable oltre al decoupling mi permette di avere altri vantaggi come il poter riutilizzare il dialogo.

Nel costruttore la prima operazione è quella di aggiornamento del record, ovviamente con la gestione del metodo scoreManager.updateIfBetter precedentemente trattato, oltre a ciò, restituisco un booleano che nella struttura del layout mostrerà la scritta “Nuovo Record” nel caso il numero di mosse appena fatto sia inferiore a quello salvato nel file che gestisce i record.

Nel layout viene impiegata una BoxLayout con i vari componenti:

Titolo → restituisce sempre Vittoria, poiché se si arriva all’EndGameDialog bisogna per forza aver accoppiato tutte le card.

Modalità scelta del game:

- Numero di mosse;

- Record attuale per la difficoltà giocata;

- Etichetta “nuovo record” in verde nel caso la condizione sia vera;

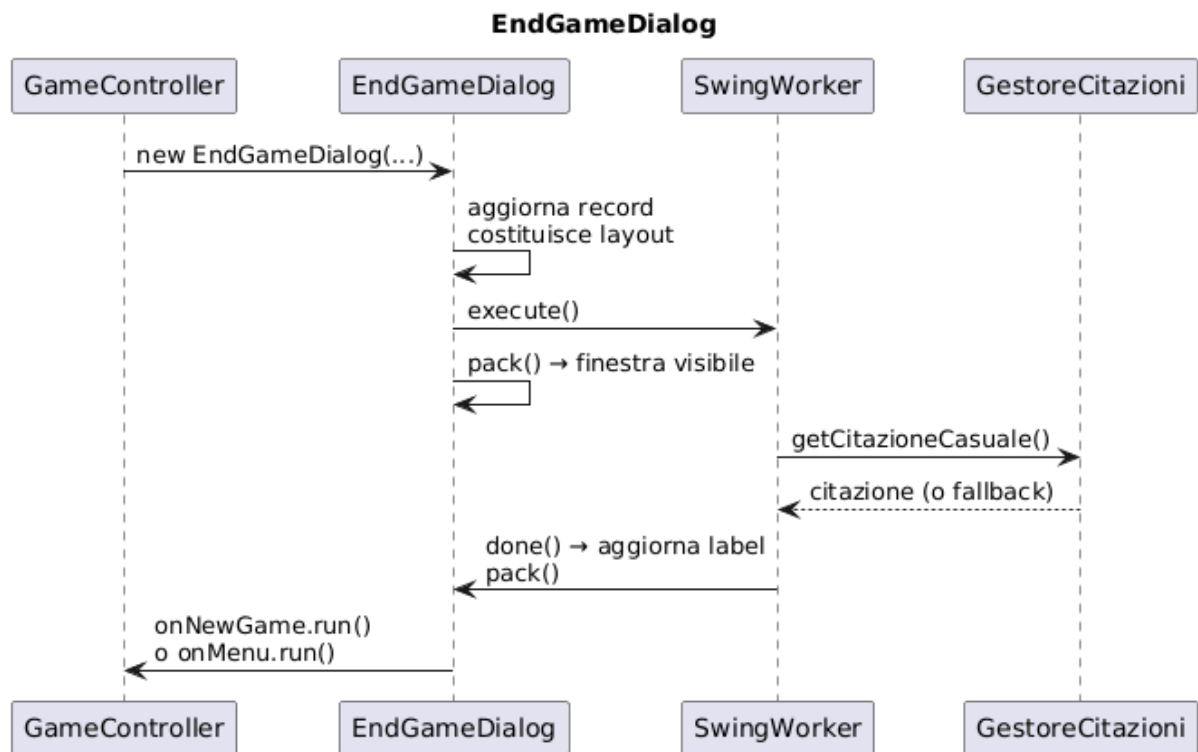
Pannello con i due pulsanti in FlowLayout, da notare che i pulsanti prima di eseguire il Runnable chiamano dispose(), questo ordine è essenziale per evitare che la finestra rimanga aperta mentre l’applicazione cambia stato.

Pack() → rende il dialogo dinamico e lo dimensiona automaticamente in base al contenuto;

SetLocationRelativo() → permette di centrare il dialogo in funzione della posizione dell’applicazione sullo schermo;

SetResizable() → impostato come false evita che ci siano ridimensionamenti che possano rompere il layout.

Il dialogo integra una frase che viene recuperata in real time attraverso `GestoreCitazioni.getCitazioneCasuale()` che a sua volta effettua una http request all'api esterna `dummyjson.com /quotes/random`.
 appena appare il dialogo non è detto che la frase sia già pronta, quando la risposta arriva il metodo `done()` aggiorna la label e richiama `pack()` che serve a ridimensionare la finestra in funzione della lunghezza della frase. in caso di errore dello `swingWorker` l'etichetta viene svuotata senza mostrare errori all'user.



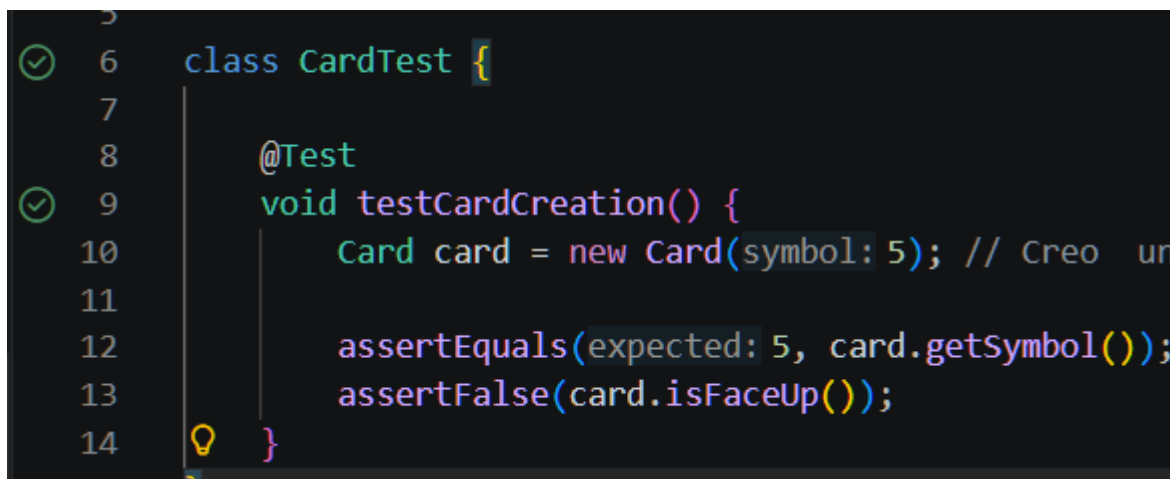
Capitolo 3 Sviluppo

3.1 Testing Automatizzato

Il progetto include una suite di test automatizzati con JUnit 5, eseguiti automaticamente dal task Gradle build. I test coprono le classi del Model, che non hanno dipendenze dalla UI e sono quindi testabili in isolamento.

Test su Card.java Marco Toma

In questa fase ho verificato tramite JUnit che la creazione di ogni singola carta avvenga correttamente. Il test controlla che il simbolo assegnato sia quello atteso e che, all'inizio della partita, la carta sia impostata correttamente come coperta (isFaceUp deve essere falso).



```
5
6 class CardTest {
7
8     @Test
9     void testCardCreation() {
10         Card card = new Card(symbol: 5); // Creo un
11
12         assertEquals(expected: 5, card.getSymbol());
13         assertFalse(card.isFaceUp());
14     }
15 }
```

Test su Game.java Marco Toma

In questa fase ho testato la logica di avanzamento del gioco e la condizione di vittoria. Il test simula il ritrovamento delle coppie e l'incremento delle mosse, verificando che lo stato della partita passi a "conclusa" (isGameOver) solo nel momento in cui l'ultima coppia necessaria viene effettivamente trovata.

```
5
6 class GameTest {
7
8     @Test
9     void testGameLogic() {
10         // coppie della difficoltà media invece di un numero fisso
11         Game game = new Game(Difficulty.MEDIUM.totalPairs());
12
13         //Aggiungo una mossa
14         game.addMove();
15         assertEquals(expected: 1, game.getMoves());
16
17         //Trovo una coppia
18         game.addMatchedPair();
19         assertEquals(expected: 1, game.getMatchedPairs());
20
21         //La partita non deve essere finita
22         assertFalse(game.isGameOver());
23     }
24
25     @Test
26     void testWinCondition() {
27         //Per il test di vittoria possiamo anche tenere un numero piccolo,
28         // ma è più elegante usare la difficoltà EASY
29         Game smallGame = new Game(Difficulty.EASY.totalPairs());
30         int pairsToWin = Difficulty.EASY.totalPairs();
31
32         //Troviamo tutte le coppie tranne l'ultima
33         for (int i = 0; i < pairsToWin - 1; i++) {
34             smallGame.addMatchedPair();
35             assertFalse(smallGame.isGameOver(), "La partita non dovrebbe finire alla coppia " + (i+1));
36         }
37     }
38 }
```

Test su Board.java Manuel Tomasello

Il boardTest verifica il corretto funzionamento di Board attraverso due test JUnit, il primo è:

TestBoardInitialiazition() → viene creato un board con difficoltà “Medium” (6x6) e verifico che la board sia esattamente della dimensione passata dall’enum Difficulty e che tutte le carte siano coperte all’inizio.

```
9      @Test
10     void testBoardInitialization() {
11         //Board diff media
12         Board board = new Board(Difficulty.MEDIUM);
13         // la dimensione del board deve essere 36 nella casistica media
14         assertEquals(expected: 36, board.getSize());
15         //Tutte le carte devono essere coperte all'inizio
16         for (int i = 0; i < board.getSize(); i++) {
17             assertFalse(board.getCard(i).isFaceUp());
18         }
19     }
20     //...
21 }
```

TestBoardSizelsEven() → verifica che il numero totale di carte sia sempre pari, per questo in questo caso ho testato sempre l’enum “Medium” ma il test funziona sia con easy sia con hard questo perché il gioco non può funzionare se c’è una carta non abbinata ad un’altra.

```
18     }
19 }
20 //carte pari
21 @Test
22 void testBoardSizeIsEven() {
23     assertEquals(expected: 0, new Board(Difficulty.MEDIUM).getSize() % 2);
24 }
25 }
```


3.2 Note di Sviluppo

3.2.1 Marco Toma

- Utilizzo della programmazione asincrona in Swing tramite `javax.swing.Timer`: Per gestire il ritardo visivo nella chiusura delle carte errate senza bloccare l'interfaccia grafica, ho dovuto studiare il funzionamento dell'Event Dispatch Thread (EDT) di Java Swing. Inizialmente, l'uso di metodi bloccanti tradizionali (come `Thread.sleep()`) causava il "congelamento" dell'intero programma. Per ovviare al problema, ho implementato un `javax.swing.Timer` che mi ha permesso di schedulare l'esecuzione asincrona del task di copertura delle carte dopo 500ms, garantendo che la GUI rimanesse sempre fluida, reattiva e pronta a registrare nuovi input.

Permalink: <https://github.com/manueltomasello/PSS25-memorygame/blob/ef7bb5987ffe8d84038ba03858c3a7e42576665e/src/main/java/it/unibo/memory/controller/GameController.java#L79>

3.2.2 Manuel Tomasello

- Salvataggio del Record (I/O su File): Ho aggiunto una funzione per salvare il miglior punteggio in un file di testo. Così, anche se si chiude il programma, il record rimane memorizzato. Ho dovuto imparare a gestire i file in Java e a usare i blocchi try-catch per evitare che il programma si chiudesse in caso di errori con il file.

Permalink: <https://github.com/manueltomasello/PSS25-memorygame/blob/dbc35609f35b4c82982499900c5a1543b248c9c9/src/main/java/it/unibo/memory/util/ScoreManager.java#L55-L61>

- Gestione dinamica della chiusura (Runnable): Per la finestra di fine partita ho usato l'interfaccia Runnable. Questo mi ha permesso di dire alla finestra 'fai questa azione' solo quando l'utente preme il tasto, senza dover scrivere troppe linee di codice ripetitive e rendendo la finestra riutilizzabile.

Permalink: <https://github.com/manueltomasello/PSS25-memorygame/blob/dbc35609f35b4c82982499900c5a1543b248c9c9/src/main/java/it/unibo/memory/view/EndGameDialog.java#L79-L81>

Capitolo 4 — Commenti Finali

4.1 Autovalutazione e Lavori Futuri

4.1.1 Marco Toma

Lo sviluppo di questo progetto ha rappresentato una sfida significativa, essendo la mia prima esperienza concreta di programmazione in un contesto così strutturato. Il percorso è iniziato dallo studio approfondito del materiale didattico, cercando di applicare i concetti di semplicità e pulizia del codice richiesti dal docente, evitando complessità superflue.

Le difficoltà principali sono state legate alla gestione del tempo, dovendo conciliare lo studio con gli impegni familiari, e all'apprendimento di strumenti nuovi come GitHub, che si è rivelato fondamentale per la collaborazione. Lavorare in team con Manuel è stato costruttivo, sebbene inizialmente abbiamo dovuto calibrare il modo di intervenire sul codice altrui per evitare conflitti.

Mi ritengo molto soddisfatto del lavoro svolto sulla classe *Card* e sulla logica del *Controller*, in particolare per la gestione asincrona del Timer. Sento di aver dato il massimo, utilizzando la stesura di questa relazione come ultimo strumento per consolidare la comprensione dell'intera architettura del software.

4.1.2 Manuel Tomasello

In questo progetto mi sono occupato principalmente di costruire l'interfaccia del gioco e di fare in modo che i punteggi venissero salvati. All'inizio è stato difficile capire come usare i Layout di Swing per mettere i bottoni e i titoli al posto giusto, ma facendo varie prove ho capito come rendere tutto più ordinato.

La parte più stimolante è stata creare la finestra finale della partita: ho imparato a usare i Runnable per far sì che, cliccando sui tasti, il gioco capisse se ricominciare o tornare al menu senza 'incartarsi'. Anche usare Git con Marco è stata un'esperienza nuova; all'inizio abbiamo avuto qualche problema a unire i nostri codici (conflict), ma poi abbiamo imparato a organizzarci meglio. In generale, questo lavoro mi ha aiutato molto a capire come si costruisce un'applicazione completa da zero e come far collaborare la parte grafica con la logica del gioco.

4.2 Difficoltà Incontrate e Commenti per i Docenti (Opzionale)

Durante lo sviluppo sono state incontrate le seguenti difficoltà principali:

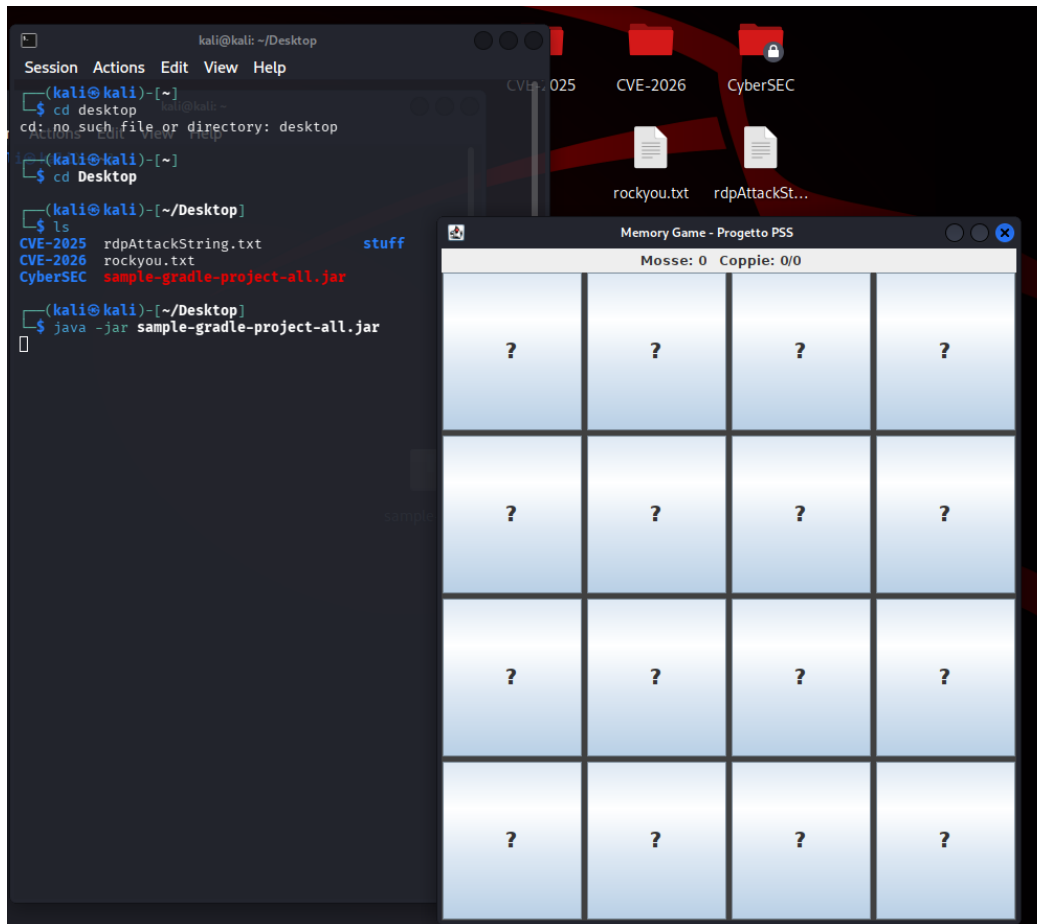
- **Compatibilità UTF-8 tra Windows e Linux:** durante la fase di testing cross-platform è emersa un'incompatibilità nell'encoding dei caratteri speciali tra Windows e Ubuntu. La soluzione è stata aggiungere la configurazione esplicita dell'encoding nel file `build.gradle.kts`.
- **Dichiarazione utilizzo AI:** Seguendo le indicazioni fornite a lezione, dichiariamo di aver utilizzato strumenti basati su LLM (Large Language Models) come supporto durante il progetto. L'intelligenza artificiale è stata impiegata come "tutor virtuale" per chiarire concetti teorici su librerie specifiche (es. Java Swing), come ausilio per la corretta sintassi di generazione dei diagrammi PlantUML e in alcuni casi come supporto diretto alla scrittura di porzioni di codice, successivamente integrate, comprese e adattate da noi. Come si evince dalla storia dei commit, non riteniamo corretto nascondere questo aspetto, tutta la logica di business e le scelte architetturali (MVC) sono state interamente ideate e scritte da noi. L'AI ha accelerato i tempi di implementazione ma non ha sostituito la comprensione del codice prodotto.
- **Gestione del versioning e Merge Conflicts:** Essendo il nostro primo progetto collaborativo strutturato, abbiamo inizialmente incontrato difficoltà nella gestione del repository Git. Lavorando in parallelo, ci siamo scontrati con dei "merge conflict" quando modificavamo file condivisi. Questo ci ha obbligato a migliorare il coordinamento e a imparare a dividere meglio i task e i branch per garantire l'integrità del codice.

Appendice A — Guida Utente

Per avviare il gioco, scaricare il file PSS25-memorygame.jar dal repository GitHub ed eseguire il seguente comando da terminale:

```
java -jar PSS25-memorygame.jar
```

Non sono necessarie installazioni aggiuntive, configurazioni o file esterni. Il software è auto contenuto nel JAR, test di esecuzione in ambiente unix:



Avvio e selezione difficoltà

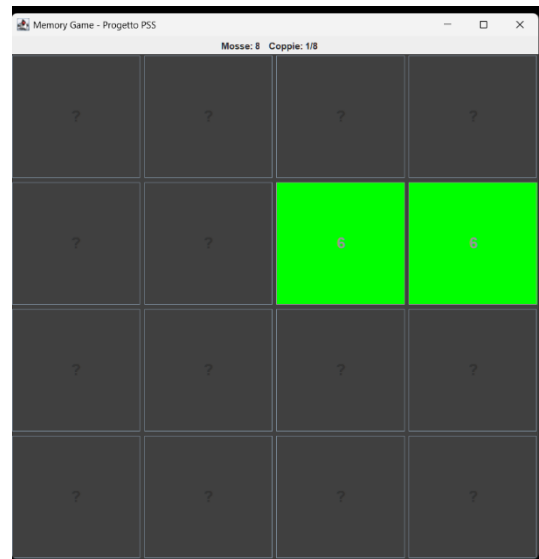
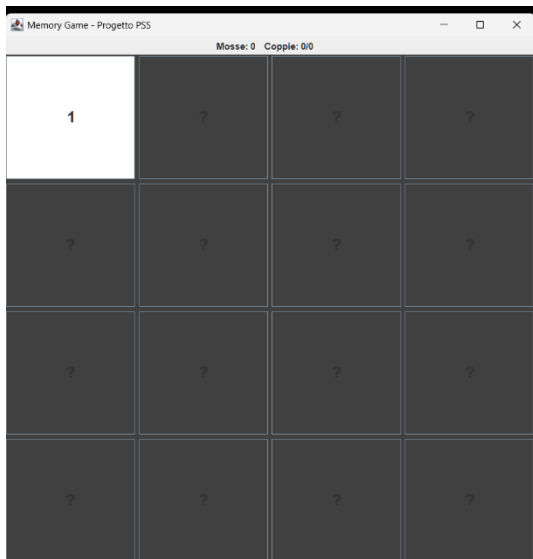
All'avvio compare il menu principale con la selezione del livello di difficoltà:

- Facile — griglia 4×4, 8 coppie da trovare
- Media — griglia 6×6, 18 coppie da trovare
- Difficile — griglia 8×8, 32 coppie da trovare



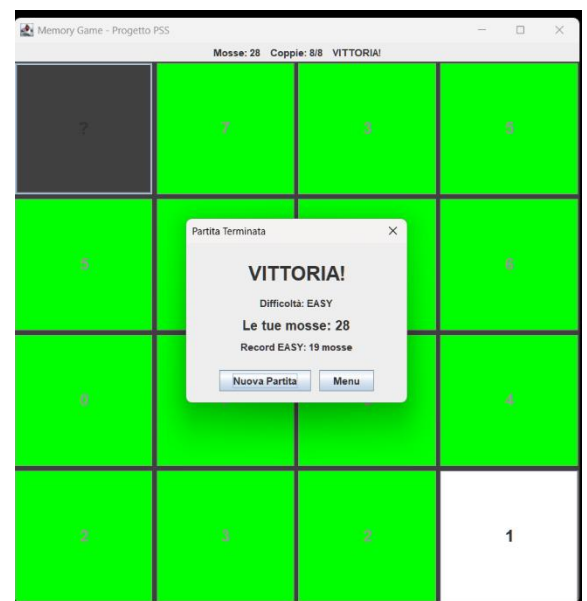
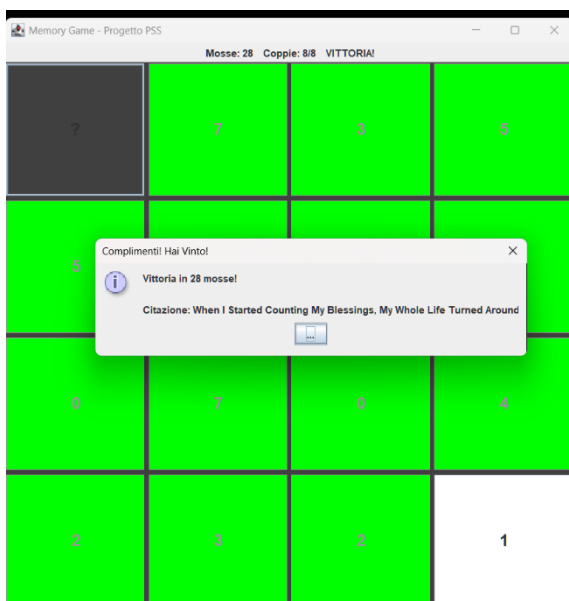
Svolgimento della partita

Fare clic su una carta per girarla. Fare clic su una seconda carta: se i simboli coincidono rimangono scoperte, altrimenti si richiudono automaticamente dopo circa mezzo secondo. Il pannello in alto mostra in tempo reale il numero di mosse e le coppie trovate.



Fine partita

Quando tutte le coppie sono trovate compare una finestra di congratulazioni con il numero totale di mosse e una citazione casuale. Chiudendo la finestra è possibile avviare una nuova partita.



Appendice B – Consuntivo Ore

A.0.1 Marco Toma

Settimana / Periodo	Attività	Ore
Marzo (Settimane 3-4)	Studio materiale didattico, analisi requisiti. Definizione Modello e classi Card e Game.	12:00
Marzo (Settimana 5)	Sviluppo logica di confronto, gestione dei turni e stato delle carte.	08:30
Aprile (Settimana 1)	Progettazione dell'interfaccia GamePanel e scrittura Test JUnit.	08:00
Aprile (Settimana 2)	Implementazione e calibrazione javax.swing.Timer per gestione ritardi (500ms).	07:00
Aprile (Settimana 3)	Integrazione logica-grafica e prima fase di bug fixing sul flusso di gioco.	08:30
Aprile (Settimana 4)	Studio intensivo: gestione concorrenza GUI (Event Dispatch Thread).	09:00
Maggio (Settimane 1-2)	Refactoring, stesura bozza relazione, revisione permalink e formattazione UML.	08:00
15-16 Maggio	Revisione finale documento, check conformità template ed esportazione PDF, correzione codice.	06:00
Totale		66:00

A.0.2 Manuel Tomasello

Settimana / Periodo	Attività	Ore
Marzo (Settimane 3-4)	Analisi requisiti iniziali e progettazione teorica del Modello di dominio.	14:00
Marzo (Settimana 5)	Implementazione della logica di gioco, gestione struttura del mazzo.	08:00
Aprile (Settimana 1)	Progettazione e sviluppo dei componenti visivi MenuPanel e StatoPanel.	06:00
Aprile (Settimana 2)	Sistema di persistenza record (ScoreManager) tramite I/O su file.	08:30
Aprile (Settimana 3)	Sviluppo EndGameDialog e gestione disaccoppiamento tramite Runnable.	08:30
Aprile (Settimana 4)	Studio e generazione dei diagrammi UML (Classe e Sequenza).	10:00
Maggio (Settimane 1-2)	Bug fixing finale, stesura autovalutazione e recupero permalink GitHub.	07:00
15-16 Maggio	Studio condiviso diagrammi UML per l'orale e rilettura incrociata relazione, mvc error fixing.	05:00
Totale		67:00